KharchaSplit API

Backend REST API Reference — v1

Backend package version: 1.3.0
Base URL: https://api.kharchasplit.com/api/v1
Document generated: 2026-05-20

KharchaSplit API Reference

KharchaSplit Backend — API Documentation

Express + PostgreSQL REST API that powers the KharchaSplit Flutter app. Backend package version `1.3.0`. This document is generated from the live route + controller code under `backend/src/`.

---

1. Conventions

1.1 Base URL

```
http://<host>:<port>/api/<version>
```

- `version` is `process.env.API_VERSION` (defaults to `v1`).
- Production deployments terminate TLS at the proxy. In production the server force-redirects plaintext to HTTPS (308) and sets HSTS for one year.

1.2 Versioning

A single major version (`v1`) is mounted in `src/server.js`. There is no `Accept-Version` header — version lives in the path.

1.3 Authentication

- Bearer JWT in the `Authorization` header on every protected endpoint: `Authorization: Bearer <accessToken>`.
- The middleware (`src/middleware/auth.js`) verifies the token with `JWT_SECRET`, looks up the user, and attaches `req.user = { id, phone_number, name, email }`.
- User lookup is cached (`auth:user:<userId>`) so it doesn't hit Postgres on every authenticated request.

1.4 Token lifecycle

Token	Purpose	Lifetime	Storage on server
Access token	Bearer auth on each call	short	not stored
Refresh token	Mint new access tokens	~30 days	`refresh_tokens` row (per device)

When the access token expires (401), the client should call `POST /auth/refresh` with its refresh token to get a new access token.

1.5 Response envelope

Every JSON response uses this shape:

Success

```
{
  "success": true,
  "message": "Optional human-readable message",
  "data": { ... }            // shape varies per endpoint; may be array
}
```

Paginated list endpoints add a `pagination` block:

```
{
  "success": true,
  "data": [ ... ],
  "pagination": {
    "page": 1,
    "limit": 50,
    "total": 137,            // not all endpoints include `total`
    "hasMore": true
  }
}
```

Error

```
{
  "success": false,
  "error": "Human readable error",
  "details": [               // only on validation failures
    { "field": "phoneNumber", "message": "Invalid phone number format" }
  ]
}
```

In non-production environments, 5xx errors also include a `stack` field.

1.6 Currency and amounts

- Amounts are JSON numbers (e.g. `123.45`), with up to two decimals.
- Currency is the ISO-4217 code (e.g. `INR`, `USD`). Symbols (`₹`) are **not** accepted — the client normalizes before sending.
- Different expenses inside one group may have different currencies; the backend does not perform FX conversion.

1.7 IDs

All entity IDs are PostgreSQL UUIDs (`uuid_generate_v4()` server-side).

1.8 Soft deletes

Most tables use `deleted_at TIMESTAMP NULL`. A `DELETE` endpoint sets `deleted_at = NOW()` rather than removing the row, so historical totals stay reproducible. Restoration is not exposed via the API.

1.9 Rate limits

Bucket	Key	Window	Max	Source
Global IP limiter	client IP	1 min	200	`server.js`
OTP send / register	phone number (body)	15 min	5	`middleware/rateLimits.js`
Expense create	authenticated user id	1 min	60	`middleware/rateLimits.js`
Settlement reminder push	(caller, target, group)	6 h	1	`groupController` SQL guard

When tripped, the server returns `429` with `{ "success": false, "error": "..." }`.

1.10 HTTP status codes

Code	Meaning in this API
200	Success.
201	Resource created.
400	Validation failure / business rule violation / missing field.
401	Missing, invalid, or expired token.
403	Authenticated, but not allowed (e.g. not a group member / admin).
404	Resource not found.
409	Conflict — duplicate phone, unsettled balance on delete, etc.
413	Request body bigger than 10 MB (large receipt photo).
429	Rate limit exceeded.
500	Internal server error. Message is sanitized in production.
502	Upstream (Twilio / WATI) error.

1.11 Body size

`express.json({ limit: '10mb' })`. Anything over 10 MB → `413`. Receipt and profile photos are base64-encoded inline.

1.12 CORS

`CORS_ORIGIN` is a comma-separated allow-list. `*` is allowed only in non-production; production refuses to start with `*` or an empty value.

---

2. Health

`GET /health`

Public. Returns server status + DB pool + cache metrics. Not under `/api/v1`.

```
{
  "success": true,
  "message": "KharchaSplit API is running",
  "version": "v1",
  "timestamp": "2026-05-20T12:00:00.000Z",
  "pool": { "total": 5, "idle": 4, "waiting": 0 },
  "cache": { "size": 12, "hits": 100, "misses": 8 }
}
```

---

3. Auth — `/api/v1/auth`

Source: `src/routes/authRoutes.js`, `src/controllers/authController.js`.

`POST /auth/register`

Public. Creates a new user (or converts a placeholder user to a real one) and triggers an OTP. Rate-limited per phone (`otpRateLimit`).

Request

```
{
  "phoneNumber": "+919876543210",    // E.164, required
  "name": "Asha",                    // 2–255 chars, required
  "email": "asha@example.com"         // optional, must be valid email
}
```

Response — 201

```
{
  "success": true,
  "message": "User registered successfully. OTP sent to your phone.",
  "data": {
    "user": {
      "id": "uuid",
      "phoneNumber": "+919876543210",
      "name": "Asha",
      "email": "asha@example.com"
    },
    "addedGroups": [                // only present if pending invites existed
      { "groupId": "uuid", "groupName": "Goa trip" }
    ],
    "convertedFromPlaceholder": false
  }
}
```

Errors

- `409` if a real (non-placeholder) user already exists for the number.

`POST /auth/send-otp`

Public. Sends a Twilio Verify SMS to the phone. Does NOT require the user to exist — `verify-otp` will auto-create one. Rate-limited per phone.

Request

```
{ "phoneNumber": "+919876543210" }
```

Response — 200

```
{ "success": true, "message": "OTP sent successfully" }
```

Errors

- `400` Invalid phone format or Twilio rejected the number.
- `500` `OTP service not configured on the server` (Twilio env not set).
- `502` Upstream Twilio failure (with `code`).

`POST /auth/verify-otp`

Public. Verifies the OTP and returns access + refresh tokens. If the phone has no row in `users`, one is auto-created with an empty `name` and `needsProfileSetup: true`.

Request

```
{
  "phoneNumber": "+919876543210",
  "otp": "123456",                    // 4–10 chars
  "device": {                         // optional, recorded on refresh_token row
    "name": "Pixel 7",
    "platform": "android",
    "osVersion": "14",
    "appVersion": "3.0.0+15"
  }
}
```

Response — 200

```
{
  "success": true,
  "message": "Login successful",      // or "Account created"
  "data": {
    "user": {
      "id": "uuid",
      "phoneNumber": "+919876543210",
      "name": "Asha",
      "email": null,
      "profileImageBase64": null,
      "preferredCurrency": "INR"
    },
    "accessToken": "eyJ...",
    "refreshToken": "eyJ...",
    "isNewUser": false,
    "needsProfileSetup": false
  }
}
```

Errors

- `401` `Invalid OTP` or `OTP expired. Please request a new one.`

`POST /auth/refresh`

Public. Exchanges a non-expired refresh token for a new access token. Updates `last_used_at` on the refresh-token row.

Request

```
{ "refreshToken": "eyJ..." }
```

Response — 200

```
{
  "success": true,
  "data": { "accessToken": "eyJ..." }
}
```

Errors

- `401` `Invalid refresh token` or `Refresh token not found or expired`.

`POST /auth/logout`

Public. Deletes the refresh token row (if provided). Always 200.

Request

```
{ "refreshToken": "eyJ..." }       // optional
```

`POST /auth/simple-login`

Public. Login by phone number without OTP. Used in dev/test flows. Will 404 if the user does not exist.

Request

```
{ "phoneNumber": "+919876543210" }
```

Response — 200 — same shape as `verify-otp` but always `isNewUser: false`.

`GET /auth/sessions`

Private. Lists every active refresh-token row for the current user.

Optional header `X-Current-Refresh-Token: <token>` lets the client mark which session is "this device" — the raw token is never returned in the payload.

Response — 200

```
{
  "success": true,
  "data": [
    {
      "id": "uuid",
      "createdAt": "2026-05-01T10:00:00Z",
      "expiresAt": "2026-05-31T10:00:00Z",
      "lastUsedAt": "2026-05-20T12:00:00Z",
      "deviceName": "Pixel 7",
      "platform": "android",
      "osVersion": "14",
      "appVersion": "3.0.0+15",
      "ipAddress": "203.0.113.7",
      "isCurrent": true
    }
  ]
}
```

`DELETE /auth/sessions/:id`

Private. Revoke a single session. `404` if it doesn't belong to the caller.

`DELETE /auth/sessions`

Private. Revoke all sessions. Pass `{ "keepRefreshToken": "<token>" }` to keep the current device signed in.

Response — 200

```
{
  "success": true,
  "message": "Other sessions signed out",
  "data": { "revokedCount": 3 }
}
```

---

4. Users — `/api/v1/users`

Source: `src/routes/userRoutes.js`, `src/controllers/userController.js`, `src/controllers/notificationController.js`.

All endpoints require authentication. Most enforce **self-only** access (`req.user.id === :id`).

`POST /users/check-registration`

Bulk lookup — given an array of phone numbers, return which are registered users.

Request

```
{ "phoneNumbers": ["+919876543210", "+919999999999"] }
```

Response — 200

```
{
  "success": true,
  "data": {
    "registered": [
      {
        "phoneNumber": "+919876543210",
        "userId": "uuid",
        "name": "Asha",
        "email": "asha@example.com",
        "profileImage": null
      }
    ],
    "unregistered": ["+919999999999"]
  }
}
```

`GET /users/by-phone/:phoneNumber`

Lookup a single user by phone. `404` if not found.

Response — 200

```
{
  "success": true,
  "data": {
    "id": "uuid",
    "phoneNumber": "+919876543210",
    "name": "Asha",
    "email": "asha@example.com",
    "profileImage": null,
    "preferredCurrency": "INR",
    "createdAt": "...",
    "updatedAt": "..."
  }
}
```

`GET /users/:id`

Same shape as `by-phone`. Lookup by user id.

`PUT /users/:id`

Self only. Update profile fields.

Request (all optional)

```
{
  "name": "Asha M",
  "email": "asha.m@example.com",
  "profileImageBase64": "<base64 PNG/JPG>",
  "preferredCurrency": "INR"
}
```

Response — 200

```
{
  "success": true,
  "message": "Profile updated successfully",
  "data": { "id": "...", "phoneNumber": "...", "name": "...", "email": "...", "profileImage":
        "...", "preferredCurrency": "INR", "updatedAt": "..." }
}
```

Errors — `403` if updating someone else's id.

`DELETE /users/:id`

Self only. Hard delete + soft delete dependent data. `200`.

`DELETE /users/:id/deactivate`

Self only. Same effect as `DELETE` today (soft delete).

`GET /users/:id/dashboard?recentLimit=10`

Self only. Aggregates balances across every active group + recent expenses. `recentLimit` defaults to 10, capped at 50.

Response — 200

```
{
  "success": true,
  "data": {
    "youAreOwed": 1250.00,
    "youOwe": 480.00,
    "totalBalance": 770.00,
    "recentExpenses": [
      {
        "id": "uuid",
        "groupId": "uuid",
        "groupName": "Goa trip",
        "description": "Dinner",
        "amount": 1200,
        "currency": "INR",
        "category": "food",
        "paidBy": {
          "id": "uuid",
          "name": "Asha",
          "phoneNumber": "+919...",
          "profileImage": null
        },
        "splitType": "equal",
        "notes": null,
        "expenseDate": "2026-05-19",
        "createdAt": "...",
        "splits": [
          { "userId": "uuid", "userName": "Asha", "owedShare": 400, "percentage": null,
        "shares": null }
        ]
      }
    ]
  }
}
```


- `youAreOwed` and `youOwe` are NOT FX-converted — totals mix currencies.
- Soft-deleted groups + expenses are excluded.

`GET /users/:id/reports?period=month|quarter|year`

Self only. Aggregated reports. Defaults to `month`.

Window relative to `NOW()`:

- `month` — current calendar month
- `quarter` — last 3 calendar months including current
- `year` — last 12 calendar months including current

Response — 200

```
{
  "success": true,
  "data": {
    "period": "month",
    "totalSpending": 4500.00,
    "youOwe": 480.00,
    "owedToYou": 1250.00,
    "categorySpending": { "food": 2200, "travel": 1800, "other": 500 },
    "monthlySpending": { "2026-05": 4500 },
    "topCategories": [
      { "category": "food", "amount": 2200 },
      { "category": "travel", "amount": 1800 }
    ]
  }
}
```


`youOwe` and `owedToYou` are point-in-time, NOT period-windowed.

`GET /users/:id/export`

Self only. Returns a single JSON blob with profile, groups (up to all active), expenses (up to 5000), personal expenses (5000), and settlements (5000). Used by the "Download your data" UI.

Response — 200


```
{
  "success": true,
  "data": {
    "exportVersion": 1,
    "exportedAt": "2026-05-20T12:00:00Z",
    "profile": { "id": "...", "phoneNumber": "...", "name": "...", "email": "...",
        "preferredCurrency": "INR", "hasProfilePhoto": true, "createdAt": "...", "updatedAt":
        "..." },
    "counts": { "groups": 4, "expenses": 137, "personalExpenses": 22, "settlements": 8 },
    "groups": [ ... ],
    "expenses": [ ... ],
    "personalExpenses": [ ... ],
    "settlements": [ ... ]
  }
}
```

FCM token (push)

Self only.

- `PUT /users/:id/fcm-token` — body `{ "fcmToken": "..." }` (legacy single-device endpoint, still supported).
- `DELETE /users/:id/fcm-token` — clears the user's token (logout).
- `POST /users/:id/devices` — body `{ "fcmToken", "platform", "deviceName", "osVersion", "appVersion" }`. Multi-device, idempotent on token.

- `DELETE /users/:id/devices` — body `{ "fcmToken": "..." }`.

Notification preferences

- `GET /users/:id/notification-prefs` — self only.

  Returns:

  ```
  {
    "success": true,
    "data": {
      "pushEnabled": true,
      "emailEnabled": false,
      "newExpense": true,
      "groupInvite": true,
      "paymentReceived": true,
      "settlementReminder": true,
      "commentMention": true,
      "weeklySummary": false,
      "productUpdates": false,
      "updatedAt": "...|null"
    }
  }
  ```

  Returns defaults (no DB row written) if the user has never updated prefs.

- `PUT /users/:id/notification-prefs` — self only. Accept any subset of the keys above (camelCase). Non-boolean values are ignored. Upserts the row. Returns same shape as `GET`.

Notifications inbox

In-app persistent inbox for push notifications. Self only.

- `GET /users/:id/notifications?limit=30&before=<iso>` — paginated reverse-chronological. Returns up to 100. Use the `createdAt` of the last row as `before` to fetch older rows.

  Item shape:

  ```
  {
    "id": "uuid",
    "type": "SETTLEMENT_REMINDER",
    "title": "Asha is requesting ₹400",
    "body": "...",
    "data": { "groupId": "...", "fromUserId": "..." },
    "isRead": false,
    "readAt": null,
    "createdAt": "..."
  }
  ```

- `GET /users/:id/notifications/unread-count` — `{ "success": true, "data": { "count": 3 } }`.

- `PATCH /users/:id/notifications/:notifId/read` — mark one as read.
- `PATCH /users/:id/notifications/read-all` — mark all unread as read.

---

5. Groups — `/api/v1/groups`

Source: `src/routes/groupRoutes.js`, `src/controllers/groupController.js`.

All endpoints require authentication. Most require **group membership**; some require **admin role**.

`GET /groups?userId=<uuid>&page=1&limit=20`

List groups the user belongs to. The `userId` query param is the user to fetch groups for (must be the caller — see authorization model). For each group, returns members + the caller's net balance.

Response — 200


```
{
  "success": true,
  "data": [
    {
      "id": "uuid",
      "name": "Goa trip",
      "description": "Apr 2026 weekend",
      "currency": "INR",
      "coverImageBase64": null,
      "createdBy": "uuid",
      "createdAt": "...",
      "updatedAt": "...",
      "isArchived": false,
      "memberCount": 4,
      "expenseCount": 12,
      "totalExpenses": 24500,
      "myBalance": -480.00,
      "members": [
        {
          "userId": "uuid",
          "name": "Asha",
          "phoneNumber": "+91...",
          "email": null,
          "profileImage": null,
          "role": "admin",
          "joinedAt": "...",
          "addedBy": "uuid",
          "isPlaceholder": false
        }
      ]
    }
  ],
  "pagination": { "page": 1, "limit": 20, "hasMore": false }
}
```


`myBalance > 0` → others owe the caller; `< 0` → caller owes others.

`GET /groups/:id`

Caller must be a member. Returns full detail + simplified-debt `balances` array.

Response — 200

```
{
  "success": true,
  "data": {
    "id": "uuid",
    "name": "Goa trip",
    "description": "...",
    "currency": "INR",
    "createdBy": "uuid",
    "coverImageBase64": null,
    "createdAt": "...",
    "updatedAt": "...",
    "isArchived": false,
    "memberCount": 4,
    "expenseCount": 12,
    "totalExpenses": 24500,
    "members": [ ... ],
    "balances": [
      { "fromUserId": "uuid", "toUserId": "uuid", "amount": 480.00 }
    ]
  }
}
```

Errors — `403` if not a member.

`POST /groups`

Create a group. Caller becomes the admin.

Request

```
{
  "name": "Goa trip",                // required, 2–255 chars
  "description": "...",               // optional
  "coverImageBase64": "<base64>",     // optional
  "currency": "INR",                  // optional, defaults from app
  "members": [                         // optional initial members
    { "userId": "uuid", "name": "Riya", "phoneNumber": "+91...", "email": "..." }
  ]
}
```

Response — 201

```
{
  "success": true,
  "message": "Group created successfully",
  "data": { /* group row */ }
}
```

`PUT /groups/:id`

Admin only. Update name / description / cover image / currency.

```
{ "name": "...", "description": "...", "coverImageBase64": "...", "currency": "INR" }
```

Errors — `403` if not admin.

`DELETE /groups/:id`

Admin only. **Blocked (409)** if any pair of members has a non-zero outstanding balance — settle everything first.

Group members

- `GET /groups/:id/members` — member only. Returns raw rows from `group_members JOIN users` (snake_case).
- `POST /groups/:id/members` — member only. Add an existing user.

  ```
  { "userId": "uuid", "name": "Riya", "phoneNumber": "+91...", "email": "..." }
  ```

- `DELETE /groups/:id/members/:userId` — admin only, OR the caller removing themselves. Self-removal **blocked (409)** if the caller has any unsettled pairwise balance.
- `PUT /groups/:id/members/:userId` — admin only. Body `{ "role": "admin" | "member" }`.

Pending members (placeholder users invited by WhatsApp via WATI)

A pending member is a placeholder `users` row (so expenses can reference them) plus a tracking row in `pending_invites`. When the real person registers, the placeholder is converted to a real user and they keep the group + expense history.

- `GET /groups/:id/pending-members` — member only.
- `POST /groups/:id/pending-members` — member only.

```
{ "name": "Riya", "phoneNumber": "+919876543210", "email": "..." }
```

Response (201):

```
{
  "success": true,
  "data": {
    "type": "placeholder",                  // or "registered" if the phone was already a
        real user
    "member": { "userId": "...", "name": "...", "phoneNumber": "...", "email": "...",
        "isPlaceholder": true },
    "pendingInvite": { "id": "...", "phoneNumber": "...", "name": "...", "email": null,
        "inviteCount": 1, "watiStatus": "sent", "createdAt": "..." },
    "watiResult": { "success": true, "error": null }
  }
}
```

- `POST /groups/:id/pending-members/:phoneNumber/resend` — member only. Resends the WATI invite.
- `DELETE /groups/:id/pending-members/:phoneNumber` — member only.

Archive / Unarchive / Complete

Admin only. All return `{ success: true, message: "..." }`.

- `PUT /groups/:id/archive` — soft-archives, push-notifies members.
- `PUT /groups/:id/unarchive` — restores the group.
- `PUT /groups/:id/complete` — same effect as archive but emits a `groupCompleted` push.

`POST /groups/:id/remind/:userId`

Send a "you owe me ₹X" push to a member. Rules enforced by the controller:

- Caller must be a group member.
- `:userId` must be a different member.
- That member must currently owe the caller > ₹0.01 in pairwise debt.
- Rate-limited to **one reminder per (caller, target, group) every 6 hours** (`429` otherwise).

Side effects: push notification (type `SETTLEMENT_REMINDER`) and an inbox row.

---

6. Expenses — `/api/v1/expenses`

Source: `src/routes/expenseRoutes.js`, `src/controllers/expenseController.js`. All require auth + group access.

`GET /expenses?groupId=<uuid>&page=1&limit=50`

Paginated. Caller must be a member of the group.

Response — 200

```
{
  "success": true,
  "data": [ /* Expense rows from `Expense.findByGroupId` */ ],
  "pagination": { "page": 1, "limit": 50, "total": 137, "hasMore": true }
}
```

`GET /expenses/:id`

Caller must be a member of the expense's group.

`POST /expenses`

Rate-limited per user (60/min). Caller must be a member of the group.

Request

```
{
  "groupId": "uuid",                    // required
  "description": "Dinner",               // required, 1–500 chars
  "amount": 1200.00,                     // required, non-negative number
  "currency": "INR",                     // optional, defaults from group
  "category": "food",                    // optional free-form string
  "paidById": "uuid",                    // required
  "paidByName": "Asha",                  // optional, used in push notification text
  "splitType": "equal",                  // optional — "equal" | "exact" | "percentage" |
      "shares"
  "receiptBase64": "<base64>",           // optional
  "notes": "...",                         // optional
  "expenseDate": "2026-05-19",           // optional, defaults to today
  "participants": [                      // required, non-empty array
    {
      "userId": "uuid",
      "amount": 400,                     // for exact / percentage / shares this carries the
        split value
      "percentage": null,
      "shares": null
    }
  ]
}
```

Response — 201 — `data` is the created expense row.

`PUT /expenses/:id`

Only the original payer (`expense.paid_by == req.user.id`) may edit. Body accepts any subset of the create fields. If `participants` is an array, splits are re-written transactionally; otherwise splits are untouched.

Errors — `403` if caller is not the payer.

`DELETE /expenses/:id`

Only the original payer may delete.

---

7. Settlements — `/api/v1/settlements`

Source: `src/routes/settlementRoutes.js` , `src/controllers/settlementController.js` . Two-step confirmation flow: payer creates → recipient confirms.

`GET /settlements?groupId=<uuid>&page=1&limit=50`

Caller must be a group member. Paginated.

`POST /settlements`

Caller must be the payer (`fromUserId === req.user.id`).

Request

```
{
  "groupId": "uuid",
  "fromUserId": "uuid",       // must equal req.user.id
  "toUserId": "uuid",
  "amount": 480,              // non-negative
  "currency": "INR",          // optional, defaults to group's currency
  "notes": "Paid via UPI"     // optional
}
```

Response — 201 — `data` is the new settlement row (status starts `pending`).

Errors

- `403` if `fromUserId !== req.user.id` .
- `409` if a pending settlement already exists between the same two users in this group (the existing row is returned in `existingSettlement`).

`PATCH /settlements/:id/confirm`

Only the recipient (`to_user_id`) may confirm. Marks status `confirmed` and invalidates balance caches.

Errors

- `403` if not the recipient.
- `400` if already confirmed.

`DELETE /settlements/:id`

Only the payer **or** a group admin may delete.

---

8. Personal expenses — `/api/v1/personal-expenses`

Source: `src/routes/personalExpenseRoutes.js` , `src/controllers/personalExpenseController.js` . Private to each user — not visible to other group members.

`GET /personal-expenses?userId=<uuid>&page=1&limit=50`

Self only. Paginated.

`GET /personal-expenses/:id`

Self only.

`POST /personal-expenses`

```
{
  "description": "Coffee",       // required, 1–500 chars
  "amount": 250,                   // required, non-negative
  "currency": "INR",               // optional
  "category": "food",              // optional
  "receiptBase64": "<base64>",     // optional
  "notes": "...",                   // optional
  "expenseDate": "2026-05-19"      // optional
}
```

`PUT /personal-expenses/:id`

Self only. Same shape as create, all fields optional.

`DELETE /personal-expenses/:id`

Self only.

---

9. Activities — `/api/v1/activities`

Source: `src/routes/activityRoutes.js`, `src/controllers/activityController.js`. The in-app activity feed.

All endpoints require auth. Endpoints that read a user's activities require `userId === req.user.id`.

`POST /activities`

Create an activity record (rarely used directly — most activities are written server-side by the controllers).

Request

```
{
  "userId": "uuid",            // required, must equal req.user.id
  "activityType": "EXPENSE_ADDED",
  "entityType": "expense",
  "entityId": "uuid",
  "title": "...",
  "description": "...",         // optional
  "metadata": { ... },           // optional JSON
  "groupId": "uuid"             // optional
}
```

`GET /activities?userId=<uuid>&page=1&limit=50`

Self only. Returns activities + `unreadCount` at top level.

Response — 200

```
{
  "success": true,
  "data": [
    {
      "id": "uuid",
      "userId": "uuid",
      "groupId": "uuid",
      "activityType": "EXPENSE_ADDED",
      "entityType": "expense",
      "entityId": "uuid",
      "title": "Asha added \"Dinner\" (₹1200)",
      "description": "...",
      "metadata": { },
      "isRead": false,
      "createdAt": "...",
      "actorName": "Asha",
      "groupName": "Goa trip"
    }
  ],
  "pagination": { "page": 1, "limit": 50, "total": 137, "hasMore": true },
  "unreadCount": 5
}
```


`GET /activities/group/:groupId?page=1&limit=50`

Returns activities for a single group. Group membership is **not** currently enforced (controller has a TODO).

`GET /activities/unread/count?userId=<uuid>`

Self only. Returns `{ "data": { "unreadCount": 5 } }`.

`GET /activities/:id`

Self only.

`PATCH /activities/:id/read`

Marks one activity as read. Returns the updated row.

`PATCH /activities/read-all`

Body: `{ "userId": "uuid" }` (must equal `req.user.id`). Marks all as read.

`PATCH /activities/group/:groupId/read-all`

Marks all of the caller's activities for that group as read.

`DELETE /activities/:id`

Self only.

---

10. Invites — `/api/v1/invites`

Source: `src/routes/inviteRoutes.js`, `src/controllers/inviteController.js`. Lightweight invite codes used in share sheets. Distinct from `pending_invites` (which are WATI/WhatsApp invitations tied to a specific group).

All endpoints require auth.

`POST /invites/create`

Batch-create invite codes for a list of phone numbers. Each row is valid for 30 days.

Request

```
{
  "phoneNumbers": ["+919876543210", "+919999999999"],
  "context": { "groupName": "Goa trip" }     // optional, included in shareMessage
}
```

Response — 200

```
{
  "success": true,
  "data": {
    "invites": [
      {
        "phoneNumber": "+919876543210",
        "inviteCode": "A1B2C3D4",
        "shareUrl": "https://kharchasplit.com/invite/A1B2C3D4",
        "shareMessage": "Join me on KharchaSplit! ..."
      }
    ]
  }
}
```

`GET /invites/status?inviteCode=A1B2C3D4`

Returns the invite's current state (`accepted` | `expired` | `pending`)

- inviter info.

`POST /invites/accept`

Body: `{ "inviteCode": "..." }`. Marks the invite as accepted.

Errors — `404` invite not found; `400` already accepted / expired.

`GET /invites/my-invites`

Returns the caller's outbound invites + summary counts.

```
{
  "success": true,
  "data": {
    "totalInvites": 12,
    "acceptedInvites": 4,
    "pendingInvites": 7,
    "expiredInvites": 1,
    "invites": [
      { "id": "uuid", "phoneNumber": "+91...", "status": "pending", "invitedAt": "...",
        "acceptedAt": null, "inviteCode": "A1B2C3D4" }
    ]
  }
}
```

`POST /invites/resend`

Body: `{ "phoneNumber": "..." }`. Returns the existing invite's share URL/message. Does NOT mint a new code or extend the expiry.

`POST /invites/cancel`

Body: `{ "inviteCode": "..." }`. Soft-deletes the invite. `404` if the code doesn't belong to the caller.

---

11. Sync — `/api/v1/sync`

Source: `src/routes/syncRoutes.js`, `src/controllers/syncController.js`.

Bulk sync endpoint for offline replays. Currently the `CREATE` / `UPDATE` / `DELETE` handlers are stubs (`handleCreate` etc. return placeholders) — the endpoint exists to keep the offline client happy and update `sync_metadata`.

`POST /sync`

```
{
  "operations": [
    { "type": "CREATE", "table": "expenses", "data": { ... },          "recordId": "client-
        uuid" },
    { "type": "UPDATE", "table": "expenses", "recordId": "server-uuid", "data": { ... } },
    { "type": "DELETE", "table": "expenses", "recordId": "server-uuid" }
  ]
}
```

Response — 200

```
{
  "success": true,
  "message": "Sync completed",
  "data": {
    "processed": 3,
    "successful": 3,
    "failed": 0,
    "results": [ { "recordId": "client-uuid", "success": true, "id": "..." } ],
    "errors":  [ { "recordId": "...", "error": "..." } ]
  }
}
```

`GET /sync/last?userId=<uuid>`

Self only. Returns rows from `sync_metadata`.

---

12. Policies (public) — `/api/v1/policies`

Source: `src/routes/policiesRoutes.js`, `src/controllers/policiesController.js`. **Public** — no auth required.

`GET /policies/privacy`

Returns the Privacy Policy as structured sections so the client can render with its own typography.

```
{
  "success": true,
  "data": {
    "title": "Privacy Policy",
    "version": "1.0.0",
    "lastUpdatedAt": "2026-05-14",
    "intro": "...",
    "sections": [
      { "heading": "What we collect", "paragraphs": ["...", "..."] }
    ]
  }
}
```

`GET /policies/terms`

Same shape, for the Terms of Service.

---

13. Error reference

Re-stating the error shape:

```
{
  "success": false,
  "error": "Human readable message",
  "details": [ { "field": "...", "message": "..." } ]   // only on 400 validation failures
}
```

Common business errors:

Status	When it fires
401	`No token provided` / `Invalid token` / `Token expired` (auth middleware).
403	`User is not a member of this group` / `User is not an admin of this group`.
403	`Only the person who paid can edit this expense`.
403	`You can only ... your own ...` (self-only endpoints).
409	`Cannot delete group with unsettled balances. Settle all dues first.`
409	`A pending settlement already exists between these users.`
409	`You must settle all balances before leaving the group.`
409	`User with this phone number already exists`.
429	`Too many OTP requests for this number.`
429	`Too many expenses created in a short window.`
429	`You've already reminded this person in the last 6 hours.`

PostgreSQL constraint errors are remapped by the global error handler:

PG code	HTTP	Message
23505	409	`Resource already exists`
23503	400	`Invalid reference`
23502	400	`Required field missing`

---

14. Cache + invalidation (informational)

In-process LRU (`src/services/cacheService.js`). Notable keys:

- `auth:user:<userId>` — every authenticated request hydrates this.

- `group:<id>:balances`, `group:<id>:settlements`, `group:<id>:expenses` — invalidated on expense / settlement writes.

If you're seeing stale data after a mutation, check that the mutating controller calls the matching `cache.invalidate(...)` — most do.

---

15. Quick endpoint index

```
GET    /health

POST   /api/v1/auth/register
POST   /api/v1/auth/send-otp
POST   /api/v1/auth/verify-otp
POST   /api/v1/auth/refresh
POST   /api/v1/auth/logout
POST   /api/v1/auth/simple-login
GET    /api/v1/auth/sessions
DELETE /api/v1/auth/sessions
DELETE /api/v1/auth/sessions/:id

POST   /api/v1/users/check-registration
GET    /api/v1/users/by-phone/:phoneNumber
GET    /api/v1/users/:id
PUT    /api/v1/users/:id
DELETE /api/v1/users/:id
DELETE /api/v1/users/:id/deactivate
GET    /api/v1/users/:id/dashboard
GET    /api/v1/users/:id/reports
GET    /api/v1/users/:id/export
PUT    /api/v1/users/:id/fcm-token
DELETE /api/v1/users/:id/fcm-token
GET    /api/v1/users/:id/notification-prefs
PUT    /api/v1/users/:id/notification-prefs
POST   /api/v1/users/:id/devices
DELETE /api/v1/users/:id/devices
GET    /api/v1/users/:id/notifications
GET    /api/v1/users/:id/notifications/unread-count
PATCH  /api/v1/users/:id/notifications/read-all
PATCH  /api/v1/users/:id/notifications/:notifId/read

GET    /api/v1/groups
POST   /api/v1/groups
GET    /api/v1/groups/:id
PUT    /api/v1/groups/:id
DELETE /api/v1/groups/:id
GET    /api/v1/groups/:id/members
POST   /api/v1/groups/:id/members
DELETE /api/v1/groups/:id/members/:userId
PUT    /api/v1/groups/:id/members/:userId
GET    /api/v1/groups/:id/pending-members
POST   /api/v1/groups/:id/pending-members
POST   /api/v1/groups/:id/pending-members/:phoneNumber/resend
DELETE /api/v1/groups/:id/pending-members/:phoneNumber
PUT    /api/v1/groups/:id/archive
PUT    /api/v1/groups/:id/unarchive
PUT    /api/v1/groups/:id/complete
POST   /api/v1/groups/:id/remind/:userId

GET    /api/v1/expenses
POST   /api/v1/expenses
GET    /api/v1/expenses/:id
PUT    /api/v1/expenses/:id
DELETE /api/v1/expenses/:id
```

```
GET    /api/v1/settlements
POST   /api/v1/settlements
PATCH  /api/v1/settlements/:id/confirm
DELETE /api/v1/settlements/:id

GET    /api/v1/personal-expenses
POST   /api/v1/personal-expenses
GET    /api/v1/personal-expenses/:id
PUT    /api/v1/personal-expenses/:id
DELETE /api/v1/personal-expenses/:id

POST   /api/v1/activities
GET    /api/v1/activities
GET    /api/v1/activities/group/:groupId
GET    /api/v1/activities/unread/count
GET    /api/v1/activities/:id
PATCH  /api/v1/activities/:id/read
PATCH  /api/v1/activities/read-all
PATCH  /api/v1/activities/group/:groupId/read-all
DELETE /api/v1/activities/:id

POST   /api/v1/invites/create
GET    /api/v1/invites/status
POST   /api/v1/invites/accept
GET    /api/v1/invites/my-invites
POST   /api/v1/invites/resend
POST   /api/v1/invites/cancel

POST   /api/v1/sync
GET    /api/v1/sync/last

GET    /api/v1/policies/privacy
GET    /api/v1/policies/terms
```